FACOLTÀ DI SCIENZE TECNOLOGICHE E DELL'INNOVAZIONE

CORSO DI LAUREA IN INGEGNERIA INFORMATICA

Prova Finale in

PROGRAMMAZIONE

Dall'analisi dei log HTTP al rilevamento automatico di anomalie, un approccio basato su Machine Learning

RELATORE

Chiar.mo

Mauro Bellone

CANDIDATO

FEDERICO MANINI

MATR.0082200317

Anno Accademico 2022/2025

TODO

Indice

Introduzione **7**

Obiettivi 7

Metodologia 8

1 Struttura di un log HTTP e Machine Learning **9**

1.1 Il protocollo HTTP in generale 9

1.2 Log HTTP, definizione e struttura 10

1.3 Log HTTP, Datamodel per il caso studio 11

1.4 Intelligenza artificiale e machine learning 13

1.5 ML, Isolation Forest per il rilevamento di anomalie 14

1.6 Isolation Forest, principi di funzionamento 16

2 Implementazione in Python **19**

2.1 Fase di estrazione e caricamento 19

2.2 Aggiunta al dataset di attacchi artificiali 20

2.3 Ingegneria delle feature 21

2.4 Applicazione dell'algoritmo 26

2.5 Interpretazione dei risultati 28

2.6 Applicazione dell'algoritmo all'intero dataset 31

3 Conclusioni **35**

3.1 Sintesi del caso studio 35

3.2 Considerazioni finali 36

Ringraziamenti **37**

Introduzione

Obiettivi

Negli ultimi anni il traffico web ha assunto un ruolo centrale nell'infrastruttura digitale globale. Applicazioni distribuite, servizi *cloud*, piattaforme *e-commerce* e sistemi informativi aziendali si basano su architetture *client–server* che utilizzano il protocollo *HTTP* (*Hyper-Text Transfer Protocol*) come meccanismo primario di comunicazione. In tale contesto, i server web producono una quantità significativa di dati sotto forma di *log*, che rappresentano una fonte informativa preziosa per finalità di monitoraggio, sicurezza, *debugging* e analisi comportamentale.

Parallelamente, l'aumento della superficie di esposizione dei servizi *web* ha reso sempre più rilevante l'individuazione tempestiva di comportamenti anomali, accessi sospetti e pattern di traffico non conformi al normale utilizzo dell'applicazione. Tecniche tradizionali basate su regole statiche o *blacklist* manuali mostrano limiti in termini di scalabilità e adattabilità a contesti dinamici. In questo scenario, l'applicazione di metodologie di analisi automatizzata e di tecniche di *machine learning* rappresenta un approccio promettente per l'identificazione di anomalie nei log dei server web.

La presente tesi si colloca all'intersezione tra programmazione, analisi dei dati e sistemi web, con l'obiettivo di progettare e implementare un sistema prototipale per l'analisi automatica dei *log HTTP* e l'individuazione di comportamenti anomali.

L'elaborato non si limita a una trattazione teorica, ma prevede la realizzazione di un caso di studio concreto basato sulla generazione e raccolta di log reali

provenienti da un'applicazione web sviluppata e distribuita.

Metodologia

Il caso di studio presentato in questo elaborato si basa sull'analisi di un dataset di *log HTTP* già esistente, opportunamente selezionato e sottoposto a un processo di anonimizzazione e censura dei dati sensibili. In particolare, sono stati rimossi o mascherati elementi quali indirizzi IP completi, eventuali identificativi personali e informazioni riconducibili a utenti specifici, nel rispetto dei principi di minimizzazione e protezione dei dati.

Il dataset utilizzato contiene richieste *HTTP* provenienti da un'applicazione web reale e comprende sia traffico ordinario sia richieste anomale o potenzialmente malevole inserite artificialmente. Prima dell'analisi, i log sono stati sottoposti a una fase di *pre-processing*, comprendente *parsing* strutturato, normalizzazione dei campi rilevanti e trasformazione in un formato idoneo all'elaborazione tramite algoritmi di *machine learning*.

Capitolo 1

Struttura di un log HTTP e Machine Learning

1.1 Il protocollo HTTP in generale

In telecomunicazioni ed informatica *l'HyperText Transfer Protocol* è un protocollo di tipo applicativo usato come principale sistema per la trasmissione di informazioni su internet. La prima versione del protocollo, la HTTP/0.9, venne implementata da Tim Berners-Lee, allora studioso del CERN di Ginevra ed aveva come fine ultimo la condivisione di informazioni tra la comunità scientifica (Berners-Lee, 1989)

Successivamente vennero rilasciate versioni che rispondevano, man mano, alle esigenze che si manifestavano.

In breve:

- HTTP/1.0, la prima versione effettivamente disponibile del protocollo ed implementata dallo stesso Berners-Lee (Fielding et al., 1996)

- HTTP/1.1, presentato come RFC 2068 (Fielding et al., 1997) ed aggiornato come descritto nel RFC 2616 (Fielding et al., 1999)

- HTTP/2 e HTTP/3 pubblicate rispettivamente nelle RFC 7540 (Belshe et al., 2015) e RFC 9114 (Bishop, 2020)

Il protocollo HTTP, come definito nell'RFC9110, lavora al livello più alto dello standard OSI (Fielding et al., 2022) con un tipo di architettura definito client/server dove il client esegue una richiesta ed il server restituisce la risposta mandata da un altro host. Nell'uso quotidiano il client corrisponde al browser mentre il server (sia quest'ultimo virtuale o fisico) è la macchina su cui risiede il sito web.

1.2 Log HTTP, definizione e struttura

Pur non esistendo una definizione formale di *log HTTP* all'interno delle specifiche del protocollo, con tale espressione si fa riferimento al file generato dal server web che registra in maniera strutturata le richieste ricevute e le relative risposte fornite. Il log rappresenta dunque una traccia persistente dell'attività del server e costituisce uno strumento fondamentale per il monitoraggio della sicurezza, la diagnosi di errori applicativi, l'analisi delle performance e lo studio del traffico.

Il primo formato strutturato per i log web è il Common Log Format (CLF), sviluppato nell'ambito del progetto NCSA HTTPd (W3C, 1995), presso il National Center for Supercomputing Applications. Si riporta di seguito estratto dell'archivio del World Wide Web Consortium che descrive la struttura del CLF (si veda la figura 1.1).

The Common Logfile Format

The common logfile format is as follows:

remotehost rfc931 authuser [date] "request" status bytes

remotehost
Remote hostname (or IP number if DNS hostname is not available, or if DNSLookup is Off.
rfc931
The remote logname of the user.
authuser
The username as which the user has authenticated himself.
[*date*]
Date and time of the request.
"*request*"
The request line exactly as it came from the client.
status
The HTTP status code returned to the client.
bytes
The content-length of the document transferred.

Figura 1.1: W3C - Common Logfile Format

Un esempio concreto risulta così strutturato:

```
127.0.0.1 - frank [10/Oct/2000:13:55:36 -0700] "GET /
    index.html HTTP/1.0" 200 2326
```

Ogni riga risulta essere una singola richiesta HTTP e contiene informazioni relative al client, al tipo di richiesta al codice di stato restituito ed infine la dimensione della risposta.

1.3 Log HTTP, Datamodel per il caso studio

Per il caso di studio oggetto della presente tesi, ci si avvale di un dataset precostruito e pubblicamente disponibile, scaricato dalla piattaforma Zenodo (Zenodo, n. d.) , repository open-access per la condivisione di dati di ricerca.

Il dataset selezionato contiene log HTTP provenienti da un ambiente reale e già sottoposti a un processo di anonimizzazione, al fine di garantire la tutela di eventuali informazioni sensibili. Tale scelta consente di operare su dati realistici, mantenendo al contempo conformità ai principi di protezione dei dati.

Di seguito si riporta un estratto esemplificativo di una riga di log convertito in formato JSON tratta dal dataset:

```
{
    "referrer":"http://search.lib.auth.gr/Record/xxxxx",
    "request":"search.lib.auth.gr:80 66.249.34457 - -
   [01/Mar/2018:00:00:01 +0200] GET /AJAX/xxxxxxxx
    HTTP/1.1 200 491 http://search.lib.auth.gr/Record/
   xxxxx Mozilla/5.0 (compatible; Googlebot/2.1; +http
   ://www.google.com/bot.html)",
    "method":"GET",
    "resource":"/AJAX/xxxxxx",
    "bytes":"491",
    "response":"200",
    "ip":"66.249.34457",
    "useragent": "Mozilla/5.0 (compatible; Googlebot
   /2.1; +http://www.google.com/bot.html)",
    "timestamp": "2018-02-28T22:00:01.000Z",
    "log_id": "-7RfsmoBCue8G08E1FyY",
}
```

- *Referrer*: URL della pagina da cui proviene la richiesta.
- *Request*: Stringa completa del log HTTP.
- *Method*: Tipologia della richiesta (GET,POST,ecc).
- *Resource*: Path richiesto sul server.
- *Bytes*: Numero di bytes restituiti nella response.
- *Response*: Tipo di HTTP Status codice
  - 2xx - Ok.
  - 3xx - Redirect.
  - 4xx - Not found.

  - 5xx - Server error.

- *Ip*: Indirizzo IP del client che ha fatto la richiesta
- *Useragent*: Informazioni sul browser e sistema operativo oppure bot che ha effettuato la richiesta.
- *Timestamp*: Formato ISO 8601(YYYY-MM-DDThh:mm:ssZ), rappresenta date e ore in modo standardizzato. Utilizza la lettera "T" per separare la data dall'ora e la Z per UTC (International Organization for Standardization, 2019).
- *Log Id*: Chiave univoca del file JSON, non ha significato semantico lato HTTP.

Come si può intuire dal formato e dai campi del datamodel preso in esame e come accennato in precedenza, quest'ultimo risulta già sottoposto ad un preprocesso di parsing.

Per valutare la fattibilità in scenari reali, è stato convalidato che gli stessi campi strutturati possono essere estratti dalle righe di registro non elaborate utilizzando l'analisi deterministica che tuttavia esula dal fine ultimo di questa tesi.

1.4 Intelligenza artificiale e machine learning

Allo stato attuale dell'evoluzione di tutto ciò che concerne gli ambiti IT, concetti come Intelligenza Artificiale e *Machine Learning* rappresentano le innovazioni più rilevanti e diffuse essendo, queste ultime, capaci di incidere significativamente sugli aspetti produttivi.

L'intelligenza artificiale viene definita come "... l'abilità di una macchina di mostrare capacità umane quali il ragionamento, l'apprendimento, la pianificazione e la creatività; tali caratteristiche consentono all'IA la comprensione del proprio ambiente, l'elaborazione di ciò che viene percepito e l'individuazione di soluzioni per un obiettivo specifico." (Camera dei Deputati, 2021)

Questa simulazione del ragionamento umano è consentita dall'analisi e dall'elaborazione dei *Big Data*, enormi quantità di dati la cui interpretazione richiede tecnologie e metodi analitici specifici (De Mauro et al., 2016).

Nell'ambito delle intelligenze artificiali moderne (e quelle prese in esame come oggetto di questa tesi) l'analisi predittiva e quella prescrittiva assumono una importanza cruciale in quanto, la prima, si avvale della capacità di apprendere e identificare pattern nei dati e quindi di anticiparli. La seconda, fornisce indicazioni per orientare le successive azioni sulla base di quanto appreso (Hüllermeier, 2021). Il machine learning (ML) è un tipo di intelligenza artificiale che consente al computer senza essere stato esplicitamente programmato. Quest'ultimo recupera informazioni da più o meno grandi set di dati ed utilizza i pattern in essi contenuti al fine di migliorarne la comprensione (Sharifani & Amini, 2023). La metodologia avanzata di ML prevede che gli algoritmi apprendano per inferenza, senza supervisione di un programmatore e, oltre ad apprendere e ad auto adattarsi, sono in grado di comprendere e rilevare alcuni schemi che l'essere umano da solo non sarebbe in grado di riconoscere. Un esempio concreto alla portata di tutti si vive nelle piattaforme di tipo social o streaming, dove i contenuti proposti dalla piattaforma stessa sono basati sulle visualizzazioni, ricerche ed interazioni di contenuti precedentemente avvenute.

1.5 ML, Isolation Forest per il rilevamento di anomalie

Isolation Forest è un algoritmo di anomaly detection non supervisionato progettato per individuare osservazioni anomale all'interno di un dataset. L'approccio si basa sul principio di isolamento delle istanze anomale invece che sulla modellazione esplicita del comportamento normale dei dati.

L'algoritmo è stato introdotto da Fei Tony Liu, Kai Ming Ting e Zhi-Hua Zhou nell'articolo "Isolation Forest", presentato alla conferenza IEEE International Conference on Data Mining (ICDM) nel 2008 (Liu et al., 2008). L'idea centrale dell'algoritmo è che le anomalie sono generalmente rare e significati-

vamente diverse rispetto alla maggior parte dei dati osservati (Liu et al., 2012). Di conseguenza, tali istanze risultano più facili da isolare attraverso partizioni casuali dello spazio delle feature.

A differenza di molti metodi tradizionali di rilevamento delle anomalie, che costruiscono un modello del comportamento normale dei dati e identificano come anomalie le deviazioni da tale modello, Isolation Forest mira direttamente a isolare i punti anomali. Questa caratteristica rende l'algoritmo particolarmente efficiente dal punto di vista computazionale e facilmente scalabile anche su dataset di grandi dimensioni (Liu et al., 2012).

Un ulteriore vantaggio è rappresentato dalla scalabilità in presenza di dataset ad alta dimensionalità. Diversamente da molti metodi basati su distanze o densità (come *k-Nearest Neighbors o Local Outlier Factor*), Isolation Forest utilizza suddivisioni casuali su singole feature, riducendo l'impatto della cosiddetta *curse of dimensionality* (Zimek et al., 2012). Questa proprietà rende l'algoritmo adatto all'analisi di dati complessi e con numerose variabili.

Dal punto di vista computazionale, Isolation Forest risulta particolarmente efficiente. L'algoritmo utilizza una strategia di sub-sampling dei dati e costruisce alberi molto leggeri, con una complessità temporale approssimativamente lineare rispetto al numero di osservazioni. Inoltre, richiede una quantità limitata di memoria (Liu et al., 2012), caratteristica che lo rende adatto all'analisi di grandi volumi di dati, come nel caso dei log HTTP generati da sistemi web.

Infine, l'algoritmo non assume specifiche distribuzioni statistiche per le feature del dataset. Questa assenza di ipotesi restrittive consente di applicarlo efficacemente anche a dati eterogenei e con distribuzioni irregolari, situazione frequente nei contesti reali di analisi dei log (Chandola et al., 2009). Nel contesto dell'analisi dei log HTTP, queste caratteristiche rendono Isolation Forest una scelta particolarmente adatta per individuare richieste potenzialmente sospette o anomale, anche in assenza di informazioni preliminari sulle tipologie di attacco o di comportamento anomalo.

1.6 Isolation Forest, principi di funzionamento

Il funzionamento di Isolation Forest si basa sulla costruzione di una collezione di alberi binari chiamati Isolation Trees (iTree)(Liu et al., 2008). Ogni albero viene costruito selezionando casualmente un sottoinsieme dei dati e applicando, a ogni nodo, due scelte casuali:

- la selezione di una feature;
- la selezione di una soglia di split compresa tra il valore minimo e massimo della feature nel sottoinsieme corrente.

Attraverso queste suddivisioni casuali, lo spazio delle feature viene progressivamente partizionato fino a isolare le singole osservazioni. Il principio fondamentale dell'algoritmo è che le anomalie, essendo rare e distanti dalla massa dei dati, tendono a essere isolate con un numero ridotto di suddivisioni. In termini di struttura dell'albero, questo significa che tali istanze raggiungono una foglia dopo un numero relativamente piccolo di livelli. Al contrario, i punti appartenenti a regioni dense del dataset richiedono un numero maggiore di suddivisioni per essere isolati, generando quindi cammini più lunghi all'interno dell'albero (Liu et al., 2008).

Per ogni osservazione (x), viene calcolata la lunghezza media del cammino (E(h(x))) attraverso tutti gli alberi della foresta. Tale valore viene successivamente normalizzato per ottenere un punteggio di anomalia (*anomaly score*). Una formulazione comune del punteggio è la seguente:

$$s(x) = 2^{-\frac{E(h(x))}{c(n)}} \tag{1.1}$$

dove (c(n)) rappresenta una costante di normalizzazione dipendente dalla dimensione del campione (n) (Liu et al., 2008).

Interpretando questo punteggio:

- valori di (s(x)) prossimi a 1 indicano che l'istanza ha un cammino medio breve e quindi è probabilmente un'anomalia;

- valori di (s(x)) prossimi a 0 indicano invece un comportamento coerente con quello della maggioranza dei dati.

Infine, per classificare le osservazioni come normali o anomale è possibile definire una soglia sul punteggio di anomalia oppure specificare un parametro di *contamination*, che rappresenta la percentuale attesa di anomalie nel dataset. In questo modo il modello può identificare automaticamente richieste HTTP sospette o comportamenti anomali all'interno dei log analizzati (Pedregosa et al., 2011).

Questo approccio consente quindi di effettuare un rilevamento automatico delle anomalie in modo efficiente e scalabile.

Capitolo 2

Implementazione in Python

2.1 Fase di estrazione e caricamento

Per poter condurre una prima analisi esplorativa sul dataset preso in esame, il file originale *(public_v2.json)* è stato trasformato in un formato più adatto alla manipolazione e interrogazione rapida. In particolare, i passaggi principali sono stati:

- Conversione in *NDJSON*: Il dataset originale consiste in un unico oggetto JSON di grandi dimensioni, con chiavi univoche per ciascun log. Tramite il pacchetto Python *ijson* è stata eseguita quindi una operazione di parsing in streaming, generando quindi un file di tipo *NDJSON*, per il quale ogni riga rappresenta un singolo log strutturato. Per le finalità del caso studio si è deciso di mantenere il campo *log_id* per avere un riferimento univoco.

- Interrogazione tramite modulo DuckDB: Il file convertito viene quindi caricato in DuckDB, un database in-process molto efficiente per dataset di grandi dimensioni. Qui è stata effettuata una prima query di test per visualizzare un sottoinsieme di dati (limitato a 1000 righe) per velocizzare la fase esplorativa.

Osservando l'output prodotto dalla funzione df.info() di Pandas (libreria fondamentale nella data analysis e data exploration), emerge subito un primo det-

taglio degno di nota. Sui primi 1000 risultati abbiamo 2 log che non hanno un method o una resource.

```
<class 'pandas.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 10 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   referrer   1000 non-null   str
 1   request    1000 non-null   str
 2   method     998 non-null    str
 3   resource   998 non-null    str
 4   bytes      1000 non-null   str
 5   response   1000 non-null   str
 6   ip         1000 non-null   str
 7   useragent  1000 non-null   str
 8   timestamp  1000 non-null   str
 9   log_id     1000 non-null   str
dtypes: str(10)
memory usage: 78.3 KB
```

Figura 2.1: Output della funzione pd.DataFrame.info

Da una indagine più approfondita si constata che il server ha restituito response di tipo 400 (*Bad Request*) e 408 (*Request Timeout*), imputabili a cause di natura sia client-side, come richieste malformate o incomplete, sia server-side, come timeout dovuti a congestione o ritardi nella gestione della connessione, comportando quindi la mancata registrazione di alcuni campi essenziali come method e resource. Questa premessa ci permetterà una analisi più approfondita in seguito.

2.2 Aggiunta al dataset di attacchi artificiali

Al fine di testare la capacità di rilevamento del sistema, il dataset è stato integrato con dei record sintetici appositamente progettati per rappresentare minacce web esplicite. Tali istanze, pur mantenendo la coerenza strutturale con il traffico legittimo presente nel dataset, presentano anomalie semantiche riconducibili a pattern di attacco noti.

Di seguito si riporta un esempio di record configurato per simulare un attacco di tipo Cross-Site Scripting (XSS):

```
{
  "referrer": "http://search.lib.auth.gr/Search/<script>
   alert('XSS')</script>",
  "request": "search.lib.auth.gr:80 185.22.12.44 - - [01/
   Mar/2018:00:00:15 +0200] GET /Search?q=<script>alert('
   XSS')</script> HTTP/1.1 400 1200",
  "method": "GET",
  "resource": "/Search?q=<script>alert('XSS')</script>",
  "bytes": "1200",
  "response": "400",
  "ip": "185.22.12.44",
  "useragent": "Mozilla/5.0",
  "timestamp": "2018-02-28T22:00:15.000Z",
  "log_id": "attack_xss"
}
```

Listing 2.1: Esempio di log JSON con attacco XSS

Per facilità di lettura, questi record sono stati catalogati con la voce *attack* nel campo *log_id* in maniera tale da rendere più facile, in un successivo momento, l'estrazione degli stessi.

2.3 Ingegneria delle feature

Al fine di rendere il dataset interpretabile dall'algoritmo di Isolation Forest, quest'ultimo è stato sottoposto ad un processo di estrazione di nuove caratteristiche (*features*). L'obiettivo è trasformare i log testuali in indicatori numerici e booleani che quantifichino anomalie strutturali, sintattiche e comportamentali nel traffico di rete.

Le feature prese in considerazione e riportate di seguito possono essere raggruppate sostanzialmente in tre macrocategorie: Analisi dimensionale, pattern matching e firme di attacco, analisi statistica e comportamentale

1. Analisi dimensionale.

   Le anomalie nelle dimensioni indicano spesso tentativi di *buffer overflow* o iniezioni di *payload* estesi.

   1.1 `resource_len` (Lunghezza della risorsa):

   Descrizione: Calcola il numero di caratteri che compongono la stringa della risorsa richiesta (*URI*).

   Razionale: Le richieste benigne tendono a raggrupparsi attorno a lunghezze standard (Kruegel & Vigna, 2003). Poiché l'algoritmo Isolation Forest è progettato per identificare le anomalie isolando i campioni rari e dimensionalmente distanti (*outlier*), fornire in input la lunghezza esatta della risorsa permette agli alberi di separare i *payload* d'attacco con pochissimi split logici, aumentando l'efficienza computazionale e l'accuratezza del rilevamento.

   1.2 `has_suspicious_request_len` (Anomalia dimensionale della risorsa):

   Descrizione: Applica il metodo statistico del Range Interquartilico (IQR) sulla lunghezza della risorsa. Assegna il valore 0 (normale) se la lunghezza rientra nell'intervallo

   $$Q_1 - 1.5 \times \text{IQR} \leq x \leq Q_3 + 1.5 \times \text{IQR} \tag{2.1}$$

   e 1 (sospetto) in caso contrario.

   Razionale: Fornisce al modello una classificazione pre-calcolata degli outlier unidimensionali, facilitando il partizionamento dei record palesemente anomali (Kruegel & Vigna, 2003).

2. Pattern matching e firme di attacco (Analisi lessicale).

   Queste feature binarie (0/1) ricercano la presenza di sintassi associata a vettori di attacco noti o a strumenti di scansione automatizzata.

   2.1 `special_char` (Conteggio dei caratteri speciali):

   Descrizione: Conta le occorrenze di caratteri non alfanumerici (es. < > ' " % ; ( ) { } * + @).

Razionale: Un'alta concentrazione di questi caratteri devia fortemente dalla baseline del traffico normale (Kruegel & Vigna, 2003), fornendo all'Isolation Forest una metrica numerica netta per separare i payload malevoli.

2.2 `has_traversal` (Rilevamento Path Traversal):

Descrizione: : Identifica la sequenza ../ nel corpo della richiesta.

Razionale: L'utilizzo della sequenza citata identifica un tentativo di risalita delle directory del server (OWASP Foundation, 2021a) (MITRE Corporation, 2025).

2.3 `has_sql_keywords` (Rilevamento *SQL Injection - SQLi*):

Descrizione: : Verifica la presenza di parole chiave tipiche del linguaggio SQL (*union, select, insert, drop*) o tautologie (*or 1=1, –, admin*).

Razionale: attacchi informatici come SQL Injection usano delle parole chiave che tentano di manipolare le query di backend del database (OWASP Foundation, 2021b).

2.4 `has_script_tag` (Rilevamento *Cross-Site Scripting - XSS*):

Descrizione: : Ricerca il tag `<script>` all'interno della richiesta.

Razionale: la ricerca mira ad identificare l'iniezione di codice eseguibile lato client, tipica degli attacchi XSS (OWASP Foundation, 2021b).

2.5 `scanner_agent` (Rilevamento *User-Agent* malevoli):

Descrizione: Cerca stringhe associate a noti tool di vulnerability scanning o exploitation automatizzata (es. sqlmap, nikto, nmap, dirbuster, o l'uso base di curl).

Razionale: Gli strumenti di scanning automatici lasciano tracce estremamente evidenti, a meno che l'attaccante non compia sforzi deliberati per nasconderle. Ogni software che effettua richieste HTTP deve dichiararsi tramite l'header user-agent (Nikto Project, 2026) (sqlmapproject, 2026) (NMAP.ORG, 2026).

L'efficacia della feature *scanner_agent* è supportata dalla natura stessa dei tool di *vulnerability scanning*, i quali, nelle configurazioni predefinite, includono identificativi univoci all'interno dell'header *HTTP User-Agent*. Tale comportamento è ampiamente documentato sia nelle specifiche tecniche degli strumenti stessi, sia nei set di regole dei principali Web Application Firewall che utilizzano queste "firme" per la chiusura immediata delle connessioni sospette.

2.6 `is_known_bot` (Identificazione di crawler e motori di ricerca tramite keyword):

Descrizione: Questa feature esegue una scansione del campo *useragent* alla ricerca di stringhe identificative (*keyword*) riconducibili a crawler benevoli e bot di indicizzazione (es. *Googlebot, Bingbot, DotBot*).

Razionale: Durante le fasi preliminari di sperimentazione, è emerso che l'attività dei crawler generava un elevato numero di falsi positivi. Tali agenti automatizzati presentano profili di navigazione (frequenza di richiesta, esplorazione sistematica delle directory e parametri complessi) che l'Isolation Forest interpreta nativamente come anomalie. L'introduzione di questa feature permette al modello di pesare correttamente questo "rumore di fondo", distinguendo tra l'automazione finalizzata all'indicizzazione (benigna) e l'automazione finalizzata all'attacco (maligna).

3. Analisi Statistica e Comportamentale.

Queste metriche valutano il comportamento dell'utente e la risposta del server, fornendo un contesto più ampio rispetto alla singola riga di log.

3.1 `is_error` (Codici di stato HTTP di errore):

Descrizione: Flag binario che indica se il server ha risposto con un codice di errore (status code ≥ 400)

Razionale: Gli attaccanti che effettuano brute-force o fuzzing, generano un'alta percentuale di risposte HTTP 400 (*Bad Request*), 403 (*Forbidden*) o 404 (*Not Found*) (Kruegel & Vigna, 2003).

3.2 `num_params` (Complessità della Query String):

Descrizione: Conta il numero di parametri passati nella risorsa (contando il carattere =).

Razionale: Le pagine prese di mira per le iniezioni (SQLi, XSS) presentano spesso query string complesse per il passaggio dei payload (Kruegel & Vigna, 2003). Più parametri sono presenti, maggiore è la probabilità che si tratti di una injection o tentativo di manipolazione.

3.3 `request_per_ip` (Frequenza di richiesta per host):

Descrizione: Calcola il numero totale di richieste effettuate da un singolo indirizzo IP rappresentando la densità di traffico generata da un singolo client.

Razionale: Valori anomali in questa metrica sono il sintomo primario di attacchi volumetrici (MITRE, 2026b) (MITRE, 2026c) (Thottan & Ji, 2003). Mentre le feature precedenti analizzano il contenuto, questa analizza quante richieste vengono fatte da un singolo indirizzo IP, rendendolo di fatto un parametro chiave per l'algoritmo nel distinguere utenti umani da script automatici.

3.4 `request_entropy` (Entropia di Shannon della richiesta):

Descrizione: Calcola l'entropia della stringa di richiesta utilizzando la formula:

$$H(X) = -\sum_{i=1}^{n} p(x_i) \log_2 p(x_i) \tag{2.2}$$

Razionale: L'entropia misura il grado di "disordine" o casualità della stringa. Payload offuscati (es. codifica Base64), shellcode o stringhe generate algoritmicamente (DGA) presentano un'entropia significativamente maggiore rispetto alle normali URL legittime dove sono presenti parole di senso compiuto e strutture ripetitive (Lyda & Hamrock, 2007)

Nota Metodologica: Si osserva che nel dataset utilizzato per il presente caso studio, i campi testuali sono stati preventivamente sottoposti a un processo di hashing per ragioni di anonimizzazione e privacy. Poiché le funzioni di hash sono progettate per massimizzare la diffusione e produrre stringhe ad alta entropia indipendentemente dall'input originale, il valore di `request_entropy` risulta uniformemente elevato per l'intero dataset. Pertanto, sebbene tale feature sia stata inclusa per completezza metodologica e per dimostrarne il valore predittivo in scenari di monitoraggio in chiaro (dove l'offuscamento malevolo si distingue nettamente dal testo naturale), nel contesto specifico di questo esperimento il suo apporto alla capacità discriminante dell'Isolation Forest è mitigato dalla natura crittografica dei dati sorgente.

3.5 `bytes_ratio` (Rapporto di trasmferimento dati):

Descrizione: Calcola il rapporto tra i byte trasferiti in una specifica transazione e la media storica dei byte trasferiti per quella medesima risorsa.

Razionale: Mentre nel traffico ordinario la dimensione della risposta per una specifica risorsa URI tende a essere costante (bassa varianza), deviazioni significative nel volume di byte trasferiti indicano potenziali anomalie post-compromissione. Come evidenziato dal framework MITRE ATTACK nella tecnica T1020 (*Automated Exfiltration*) (MITRE, 2026a), il monitoraggio del volume cumulativo di dati permette di rilevare l'estrazione di informazioni sensibili tramite SQL Injection o accessi non autorizzati, rendendo tale rapporto un input critico per l'algoritmo di isolamento.

2.4 Applicazione dell'algoritmo

L'algoritmo di Isolation Forest opera attraverso partizioni casuali dello spazio delle feature (Liu et al., 2008); per questo motivo, il dataset è stato ridotto alle

sole componenti numeriche e booleane derivate.

Sebbene l'algoritmo sia intrinsecamente invariante alla scala, è stato scelto di integrare un MinMaxScaler all'interno della pipeline di addestramento.

Nota Metodologica: L'uso della normalizzazione in questo caso studio non è dettato dai vincoli del modello di isolamento, ma dalla necessità di stabilizzare il processo di *Hyperparameter Tuning*. Scalando le feature in un range $[0, 1]$, si garantisce che durante la fase di *Grid Search* il calcolo delle metriche di scoring avvenga su una distribuzione controllata, facilitando la convergenza verso i parametri ottimali.

Per evitare una scelta arbitraria dei parametri (come la contamination o il numero di alberi), è stata implementata una ricerca sistematica tramite *GridSearchCV*. Questo approccio trasforma un modello tipicamente non supervisionato in un sistema "guidato" dai dati reali (nel caso studio, gli attacchi inseriti manualmente).

- Scoring Strategy: È stata scelta la *recall_macro* come metrica di riferimento (`refit='rec'`). In un contesto come quello del presente caso studio, è prioritario massimizzare il richiamo (*Recall*), ovvero la capacità del sistema di non farsi sfuggire i veri attacchi, anche a costo di accettare un numero leggermente superiore di falsi positivi.

- Risultati dell'Ottimizzazione e Parametri ottimali:
  La configurazione ottimale identificata ha prodotto un Recall di $\sim$0.79 con i valori di `contamination` e `n_estimators` impostati rispettivamente a 0.01 e 1000.

Un passaggio tecnico fondamentale riguarda la mappatura delle classi per la compatibilità con la libreria Scikit-Learn. Poichè l'Isolation Forest restituisce -1 per le anomalie e 1 per il traffico normale, le etichette del Ground Truth sono state allineate in tal senso.

```
1    df['is_attack_real'] = df['log_id'].str.contains('attack').
    astype(int)
2    df.loc[df['is_attack_real'] == 1, 'is_attack_real'] = -1
```

```
    df.loc[df['is_attack_real'] == 0, 'is_attack_real'] = 1
```

Questo allineamento permette alle funzioni di scoring della Grid Search di interpretare correttamente le anomalie come la classe di interesse da rilevare.

2.5 Interpretazione dei risultati

L'algoritmo Isolation Forest non restituisce un verdetto binario deterministico, bensì un valore di "anomalia" continuo. Mentre i punteggi negativi più profondi identificano record con feature multiple "fuori scala" (ovvero campioni che richiedono pochissimi partizionamenti per essere isolati), i punteggi prossimi allo zero rappresentano una zona di incertezza statistica dove solitamente ricadono i falsi positivi.

Si riporta di seguito un primo snippet di codice e la relativa tabella per verificare come l'algoritmo abbia classificato gli attacchi inseriti artificialmente nel dataset per testare la sensibilità del modello:

```
df[df['log_id'].str.contains('attack')][['log_id', '
    anomaly_score', 'scores']]
```

Listing 2.2: Verifica classificazione attacchi artificiali

Index	log_id	anomaly_score	scores
1000	attack_xss	-1	-0.065746
1001	attack_sql	-1	-0.110561
1002	attack_traversal	-1	-0.125805
1003	attack_scanner	-1	-0.138416

Tabella 2.1: Scores per gli attacchi artificiali

Come si evince dai risultati, tutti gli attacchi iniettati sono stati correttamente identificati come anomalie (`anomaly_score = -1`), con punteggi di isolamento significativamente bassi, confermando l'efficacia delle feature estratte nel caratterizzare pattern malevoli noti.

Per convalidare il caso studio, si è proceduto al calcolo della matrice di confusione e del relativo *classification report*. Questi strumenti sono essenziali per misurare la capacità del modello di distinguere tra traffico lecito e tentativi di intrusione.

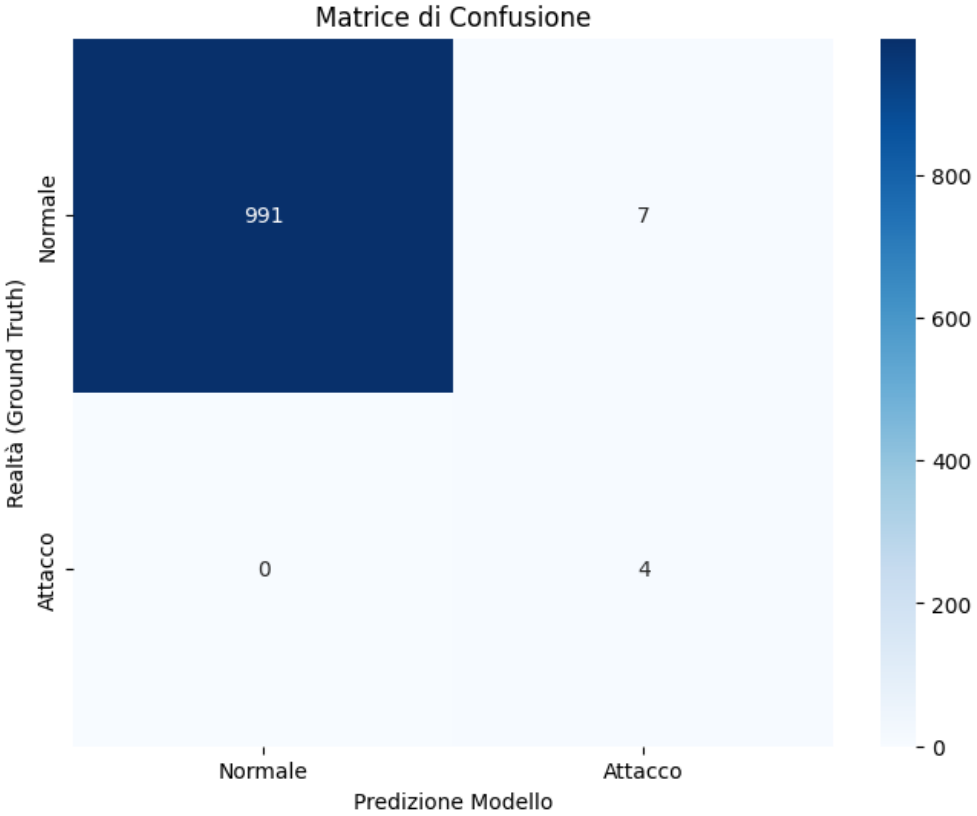

Figura 2.2: Matrice di confusione

```
from sklearn.metrics import classification_report
print(classification_report(df['is_attack_real'], df['
    is_attack_predicted']))
```

Listing 2.3: Generazione del Classification Report

...	precision	recall	f1-scores	support
0	1.00	0.99	1.00	998
1	0.36	1.00	0.53	4
accuracy	-	-	0.99	1002
macro avg	0.68	1.00	0.76	1002
weighted avg	1.00	0.99	0.99	1002

Tabella 2.2: Classification report

L'analisi del report evidenzia una **Recall** del 100% sulla classe 1 (attacchi), indicando che il modello non ha generato falsi negativi. Tuttavia, la **Precision** si attesta allo 0.36; questo valore, pur sembrando basso in un contesto di classificazione standard, è tipico dei sistemi di *Anomaly Detection*. Come sottolineato da Sommer e Paxson (Sommer & Paxson, 2010), nei sistemi di *intrusion detection* basati su anomalie l'elevata incidenza di falsi positivi rappresenta uno dei principali limiti all'adozione in ambito operativo, in quanto attività lecite ma atipiche possono facilmente essere classificate come sospette.

Oltre agli attacchi artificiali, il modello ha catalogato come anomalie altri record presenti nel traffico originale. Di seguito vengono riportati i campioni con il punteggio di isolamento più basso:

```
top_anomalies = df[df['anomaly_score']==-1][['request','scores'
    ]].sort_values(by='scores')
```

Listing 2.4: Estrazione delle top anomalie

Index	Request Log Entry
1003	search.lib.auth.gr:80 203.0.113.45 - - [01/Mar/2018:00:00:30 +0200] GET /wp-admin/install.php HTTP/1.1 404 250 - sqlmap/1.7
1002	search.lib.auth.gr:80 91.200.12.77 - - [01/Mar/2018:00:00:25 +0200] GET /../../../../etc/passwd HTTP/1.1 403 300 - curl/7.58.0
1001	search.lib.auth.gr:80 45.77.88.12 - - [01/Mar/2018:00:00:20 +0200] GET /login?user=admin' OR 1=1-- HTTP/1.1 500 800 http://evil.com Mozilla/5.0
1000	search.lib.auth.gr:80 185.22.12.44 - - [01/Mar/2018:00:00:15 +0200] GET /Search?q=<script>alert('XSS')</script> HTTP/1.1 400 1200 ...
918	search.lib.auth.gr:80 176.92.34367 - - [01/Mar/2018:00:07:56 +0200] GET /Cover/Show?author=Winstanley... HTTP/1.1 200 677
826	search.lib.auth.gr:80 176.92.34367 - - [01/Mar/2018:00:07:34 +0200] GET /Cover/Show?author=... HTTP/1.1 200 678
629	search.lib.auth.gr:80 155.207.36594 - - [01/Mar/2018:00:05:46 +0200] GET /Cover/Show?author=... HTTP/1.1 200 677
497	search.lib.auth.gr:80 176.92.58901 - - [01/Mar/2018:00:05:08 +0200] GET / HTTP/1.1 200 4709 - Mozilla/5.0
429	search.lib.auth.gr:80 66.249.58337 - - [01/Mar/2018:00:04:25 +0200] GET /themes/jquerymobile/... HTTP/1.1 200 32415
327	search.lib.auth.gr:80 85.74.35922 - - [01/Mar/2018:00:03:05 +0200] GET /themes/bootstrap3/... HTTP/1.1 200 65714
59	search.lib.auth.gr:80 141.237.44390 - - [01/Mar/2018:00:00:19 +0200] GET /themes/bootstrap3/... HTTP/1.1 200 65714

Tabella 2.3: Anomalie per punteggio di *scores*

Da un'analisi approfondita, emerge che tali richieste non appartengono al "rumore di fondo" generato dai crawler (correttamente filtrati dalla feature `is_known_bot`), ma presentano valori di `resource_len` e `bytes_num` estremamente distanti dalla media. Ad esempio, le richieste ai temi grafici (Index 327, 59) o i file CSS/JS molto pesanti vengono isolati per la loro mole di dati. Nonostante la loro natura anomala dal punto di vista statistico, essi possono essere classificati come falsi positivi in ambito di sicurezza, rappresentando traffico lecito ma strutturalmente pesante.

2.6 Applicazione dell'algoritmo all'intero dataset

Nelle sezioni precedenti, come indicato in precedenza, per effettuare una fase esplorativa e calibrare i parametri dell'algoritmo, si è proceduto a prendere in considerazione solo i primi 1000 record del dataset.

Per convalidare i risultati si è dunque applicato l'algrotimo all'intero dataset, scoprendo uno dei limiti principali dell'algoritmo di isolation forest. Degli attacchi inseriti manualmente solo uno viene riconosciuto come tale. dei circa 4 milioni di record, circa 100.000 vengono riconosciuti come anomalie e gli attacchi inseriti manualmente vengono classificati come Falsi Negativi.

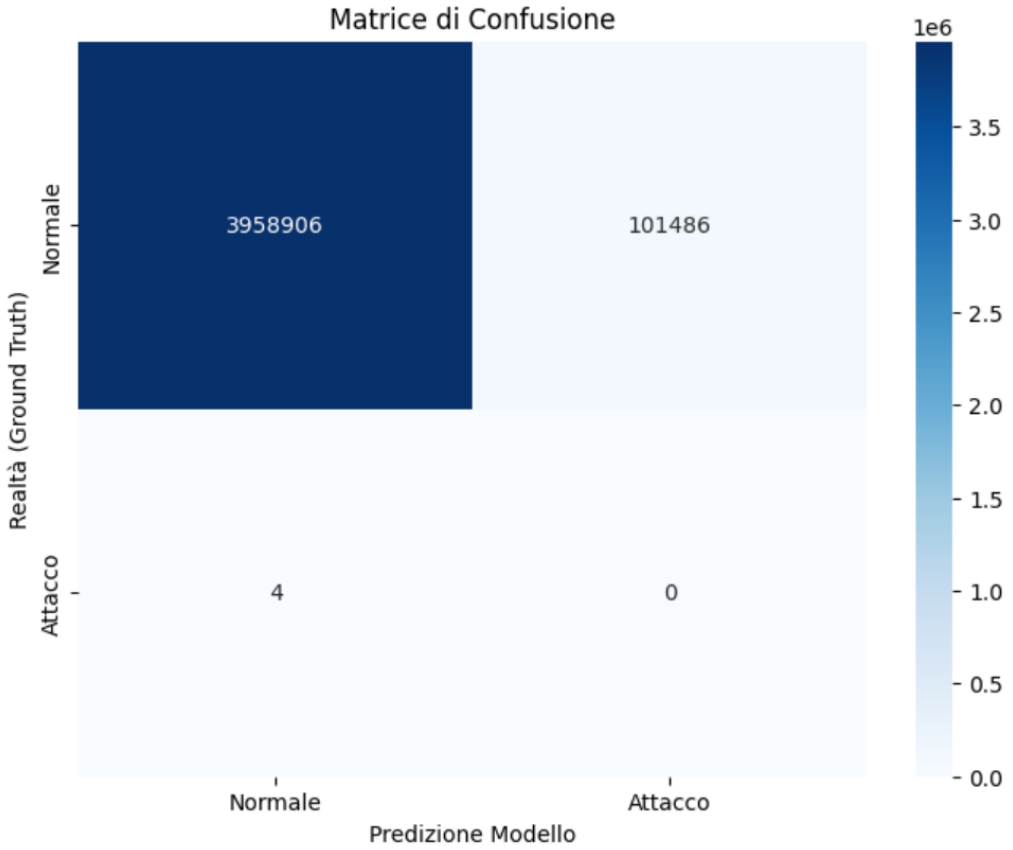

Figura 2.3: Matrice di confusione applicata all'intero dataset

Dopo una più approfondita operazione di clustering ricavato dalla seguente query:

```
1    top_anomalies = df[df["anomaly_score"] == -1].sort_values(by
    ='scores')
2    top_anomalies_melted = top_anomalies.melt(
3    id_vars='log_id',
4    var_name='categories',
5    value_vars=features_boolean,
6    value_name='value')
```

```

active = top_anomalies_melted[top_anomalies_melted["value"] ==
    1]
combos = active.groupby("log_id")["categories"].agg(lambda x: ",
     ".join(tuple(sorted(x)))).reset_index(name='tuple')
```

Si ricava la seguente tabella:

Combinazione di Feature	**Frequenza**
is_known_bot, special_chars	86.654
has_suspicious_request_len	4.033
has_suspicious_request_len, is_known_bot, special_chars	3.011
has_suspicious_request_len, is_known_bot	2.879
is_known_bot	2.556
has_suspicious_request_len, is_error	1.340
has_sql_keywords, has_suspicious_request_len, is_error	164
has_suspicious_request_len, is_error, is_known_bot	131
is_error, is_known_bot, special_chars	86
has_suspicious_request_len, special_chars	68
is_error, is_known_bot	34
has_sql_keywords, has_suspicious_request_len	15
is_error	4
has_suspicious_request_len, is_error, is_known_bot, special_chars	1

Tabella 2.4: Report di classificazione delle occorrenze per combinazione di feature.

Appare quindi evidente che il rumore di fondo generato dai web crawler offuschi quelli che l'algoritmo dovrebbe catalogare come attacchi.

Capitolo 3

Conclusioni

3.1 Sintesi del caso studio

Il presente lavoro di tesi ha avuto come obiettivo principale la progettazione, implementazione e valutazione di un sistema di *Anomaly Detection* basato sull'algoritmo Isolation Forest, applicato all'analisi di log HTTP provenienti da un contesto reale.

Il punto di partenza dell'elaborato risiede nella constatazione che i sistemi di difesa tradizionali, fondati su firme statiche e regole predefinite, mostrano limiti significativi nel fronteggiare la crescente dinamicità delle minacce informatiche e l'elevata variabilità del traffico web moderno. In tale scenario, approcci basati su tecniche di Machine Learning risultano particolarmente promettenti, in quanto permettono di individuare comportamenti anomali senza la necessità di una conoscenza preventiva delle specifiche tipologie di attacco.

L'attività svolta si è quindi concentrata sulla costruzione di un dataset derivato da log HTTP, opportunamente trasformato attraverso un processo di *feature engineering* volto a estrarre metriche significative dal punto di vista della sicurezza applicativa. Su tali dati è stato applicato un modello non supervisionato, con l'obiettivo di verificare la capacità dell'algoritmo di identificare automaticamente richieste anomale o potenzialmente malevole.

I risultati ottenuti evidenziano come l'approccio adottato sia efficace nell'individuare comportamenti anomali all'interno del traffico analizzato ma che risulti

non direzionato all'ambito cybersecurity. In particolare, il modello dimostra una buona sensibilità nel rilevare pattern atipici, confermando la validità dell'ipotesi iniziale. Tuttavia, come prevedibile nei sistemi di anomaly detection non supervisionati, emerge la presenza di falsi positivi, spesso associati a traffico lecito ma caratterizzato da elevata complessità o intensità, oscurando del tutto quelli che possono essere attacchi reali.

Tale comportamento evidenzia un compromesso intrinseco tra sensibilità e precisione: se da un lato il modello è in grado di intercettare un ampio spettro di anomalie, dall'altro richiede ulteriori meccanismi di filtraggio o validazione per ridurre il numero di segnalazioni non rilevanti a beneficio di quelli realmente rilevanti per l'utente finale. Questo aspetto rappresenta una caratteristica comune nei modelli di Machine Learning applicati alla cybersecurity, in cui il raggiungimento simultaneo di elevati livelli di precisione e accuratezza risulta particolarmente complesso, soprattutto in assenza di dati etichettati.

3.2 Considerazioni finali

Ringraziamenti

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Bibliografia

Belshe, M., Peon, R., & Thomson, M. (2015, maggio). *Hypertext Transfer Protocol Version 2 (HTTP/2)*. IETF. Recuperato febbraio 27, 2026, da https://www.rfc-editor.org/rfc/rfc7540

Berners-Lee, T. (1989). *Information Management: A Proposal*. CERN. Recuperato febbraio 27, 2026, da https://www.w3.org/History/1989/proposal.html

Bishop, M. (2020, giugno). *RFC 9114 HTTP/3*. IETF. Recuperato febbraio 27, 2026, da https://www.rfc-editor.org/rfc/rfc9114

Camera dei Deputati. (2021). *Riunione interparlamentare della Commissione speciale sull'intelligenza artificiale in un'era digitale (AIDA) del Parlamento europeo sul tema "Intelligenza artificiale e decennio digitale"* (Dossier di documentazione N. RI070) (Legge XVIII). Servizio Studi - Dipartimento UE. Recuperato marzo 15, 2026, da https://documenti.camera.it/leg18/dossier/testi/RI070.htm

Chandola, V., Banerjee, A., & Kumar, V. (2009). Anomaly Detection: A Survey. *ACM Computing Surveys (CSUR)*, *41*(3), 1–58. https://doi.org/10.1145/1541880.1541882

De Mauro, A., Greco, M., & Grimaldi, M. (2016). A formal definition of Big Data based on its essential features. *Library Review*, *65*(3), 122–135. https://doi.org/10.1108/LR-06-2015-0061

Fielding, R. T., Gettys, J., Mogul, J., Frystyk, H., & Berners-Lee, T. (1996, maggio). *Hypertext Transfer Protocol – HTTP/1.0*. Internet Engineering Task Force. Recuperato febbraio 27, 2026, da https://www.rfc-editor.org/rfc/rfc1945

Fielding, R. T., Gettys, J., Mogul, J., Frystyk, H., & Berners-Lee, T. (1997, gennaio). *Hypertext Transfer Protocol – HTTP/1.1*. IETF. Recuperato febbraio 27, 2026, da https://www.rfc-editor.org/rfc/rfc2068

Fielding, R. T., Gettys, J., Mogul, J., Frystyk, H., Masinter, L., Leach, P., & Berners-Lee, T. (1999, giugno). *Hypertext Transfer Protocol – HTTP/1.1*. IETF. Recuperato febbraio 27, 2026, da https://www.rfc-editor.org/rfc/rfc2616

Fielding, R. T., Nottingham, M., & Reschke, J. (2022, giugno). *RFC 9110, HTTP Semantics*. IETF. Recuperato febbraio 27, 2026, da https://www.rfc-editor.org/rfc/rfc9110

Hüllermeier, E. (2021). *Prescriptive machine learning for automated decision making: Challenges and opportunities*. arXiv: 2112.08268 [cs.LG]. Recuperato marzo 31, 2026, da https://arxiv.org/abs/2112.08268

International Organization for Standardization. (2019). *Date and time — Representations for information interchange — Part 1: Basic rules*. ISO. Geneva, Switzerland.

Kruegel, C., & Vigna, G. (2003). Anomaly Detection of Web-based Attacks. *Proceedings of the 10th ACM Conference on Computer and Communications Security (CCS)*, 251–261. https://doi.org/10.1145/948109.948144

Liu, F. T., Ting, K. M., & Zhou, Z.-H. (2008). Isolation Forest. *Proceedings of the 2008 Eighth IEEE International Conference on Data Mining (ICDM)*, 413–422. https://doi.org/10.1109/ICDM.2008.17

Liu, F. T., Ting, K. M., & Zhou, Z.-H. (2012). Isolation-Based Anomaly Detection. *ACM Transactions on Knowledge Discovery from Data (TKDD)*, *6*(1), 3:1–3:39. https://doi.org/10.1145/2133360.2133363

Lyda, R., & Hamrock, J. (2007). Using Entropy Analysis to Find Malware. *IEEE Security & Privacy Magazine*, *5*(2), 40–45. https://doi.org/10.1109/MSP.2007.48

MITRE. (2026a). *Techniques – Automated Exfiltration* [ID: T1020. Ultimo accesso 23/03/2026]. Recuperato marzo 23, 2026, da https://attack.mitre.org/techniques/T1020/

MITRE. (2026b). *Techniques – Brute Force* [ID: T1110. Ultimo accesso 23/03/2026]. Recuperato marzo 23, 2026, da https://attack.mitre.org/techniques/T1110/

MITRE. (2026c). *Techniques – Denial of Service* [ID: T1498. Ultimo accesso 23/03/2026]. Recuperato marzo 23, 2026, da https://attack.mitre.org/techniques/T1498/

MITRE Corporation. (2025). *CWE-22: Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal')* [Ultimo accesso 23/03/2026]. Recuperato marzo 23, 2026, da https://cwe.mitre.org/data/definitions/22.html

Nikto Project. (2026). *Config-Variables - USERAGENT* [GitHub Wiki. Ultimo accesso 23/03/2026]. Recuperato marzo 23, 2026, da https://github.com/sullo/nikto/wiki/Config-Variables

NMAP.ORG. (2026). *Script Arguments – http.useragent* [Nmap Scripting Engine (NSE) Documentation. Ultimo accesso 23/03/2026]. Recuperato marzo 23, 2026, da https://nmap.org/nsedoc/lib/http.html

OWASP Foundation. (2021a). *A01:2021 – Broken Access Control* [Ultimo accesso 23/03/2026]. Recuperato marzo 23, 2026, da https://owasp.org/Top10/A01_2021-Broken_Access_Control/

OWASP Foundation. (2021b). *A03:2021 – Injection* [Ultimo accesso 23/03/2026]. Recuperato marzo 23, 2026, da https://owasp.org/Top10/A03_2021-Injection/

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, *12*, 2825–2830. http://jmlr.org/papers/v12/pedregosa11a.html

Sharifani, K., & Amini, M. (2023). Machine learning and deep learning: A review of methods and applications. *World Information Technology and Engineering Journal*, *10*(07), 3897–3904. http://www.witej.com/

Sommer, R., & Paxson, V. (2010). Outside the Closed World: On Using Machine Learning for Network Intrusion Detection. *2010 IEEE Symposium on Security and Privacy*, 305–316. https://doi.org/10.1109/SP.2010.25

sqlmapproject. (2026). *Usage – HTTP User-Agent header* [GitHub Wiki. Ultimo accesso 23/03/2026]. Recuperato marzo 23, 2026, da https://github.com/sqlmapproject/sqlmap/wiki/Usage

Thottan, M., & Ji, C. (2003). Anomaly detection in IP networks. *IEEE Transactions on Signal Processing*, *51*(8), 2191–2204. https://doi.org/10.1109/TSP.2003.814751

W3C. (1995, luglio). *Logging Control In W3C httpd – The Common Logfile Format*. World Wide Web Consortium. Recuperato febbraio 27, 2026, da https://www.w3.org/Daemon/User/Config/Logging.html#common-logfile-format

Zenodo. (n. d.). *Web robot detection - Server logs*. Zenodo. Recuperato marzo 2, 2026, da https://zenodo.org/records/3477932

Zimek, A., Schubert, E., & Kriegel, H.-P. (2012). A survey on unsupervised outlier detection in high-dimensional numerical data. *Statistical Analysis and Data Mining: The ASA Data Science Journal*, *5*(5), 363–387. https://doi.org/10.1002/sam.11161